The VLDB Journal manuscript No.
(will be inserted by the editor)

LETI: Learned Space Filling Curves for Efficient Trajectory Indexing in Key-Value Databases

Tianyu Huang[1] · **Yingqi Tang**[1] · **Ruiyuan Li**[1] · **Guangchao Yang**[1] · **Huajun He**[2] · **Cheng Long**[3] · **Yu Zheng**[2,4]

Abstract Large-scale trajectory data are increasingly managed in key-value (KV) stores, where query processing is dominated by range scans over keys followed by refinement. In this setting, the design of trajectory keys directly determines query efficiency. Existing methods rely on full spatial hierarchies, predefined or workload-specific space-filling curves, and globally fixed shape granularities, which often lead to fragmented scan ranges and excessive candidates, especially under skewed and heterogeneous trajectory distributions. We propose LETI, a trajectory indexing framework that jointly optimizes spatial hierarchy, traversal order, and shape granularity. LETI constructs a compact quadtree to eliminate redundant sparse regions, learns a traversal order that improves scan locality, and adapts shape partitions according to local trajectory distributions to enhance filtering effectiveness. This joint design produces more compact scan ranges and reduces the number of candidates passed to refinement. Experiments on real-world datasets show that LETI substantially reduces query latency and improves filtering effectiveness across diverse workloads. Overall, LETI outperforms the best-performing baseline by up to 45.7%.

Keywords Trajectory indexing · Key-value databases · Learned index · Space-filling curves · Reinforcement learning

Corresponding author: ruiyuan.li@cqu.edu.cn.

[1]College of Computer Science, Chongqing University, Chongqing, China
[2]Southwest Jiaotong University, Chengdu, China
[3]Nanyang Technological University, Singapore
[4]Xidian University, Xi'an, China
E-mail: ruiyuan.li@cqu.edu.cn

1 Introduction

Trajectories are a fundamental form of spatio-temporal data that record object movement as sequences of time-ordered locations [46]. With the proliferation of mobile devices, location-based services, and Internet of Things (IoT) systems, large-scale trajectory collections have become a key data foundation for a wide range of applications, including trajectory pattern mining [7], path prediction [18], and point-of-interest (POI) discovery [39]. As both the volume and importance of trajectory data continue to grow, efficient indexing and query processing have become a central system problem. Key-value (KV) stores are well suited for this setting due to their high write throughput, low access latency, and horizontal scalability [24]. In KV-based trajectory management systems, trajectories (or trajectory segments) are indexed through keys, while their detailed spatio-temporal contents are stored as values. As a result, query processing is largely reduced to range scans over keys followed by refinement, making key design a critical factor in query efficiency.

Space-filling curves (SFCs) are widely used to encode multi-dimensional spatial locations into one-dimensional values. An SFC (e.g., Z-order [31] curve) recursively partitions the spatial domain into hierarchical cells and assigns each cell a one-dimensional code according to a predefined traversal order. Unlike point objects, trajectories typically span multiple cells and therefore cannot be represented by a single cell code. To better approximate the spatial extent of a trajectory, XZ-order [5] indexes a trajectory based on its minimum bounding rectangle (MBR) (cf. Definition 1). Consider the hierarchical cells that contain the MBR's bottom-left corner. Among them, the smallest cell whose enlarged element (abbr. EE, the region expanded by doubling cell's width and height) (cf. Definition 4) fully covers the MBR is

selected, and its corresponding SFC code is used as the trajectory's **spatial encoding**. As shown in Fig. 1a, the MBRs of two trajectories are both covered by the EE of the same cell, labeled '00', and thus share the same spatial encoding. Some studies [14, 15, 22, 23] further incorporate **shape encoding** into XZ-ordering. They partition each EE into equal-sized subcells and encode the trajectory according to the subcells it intersects. This finer-grained representation helps distinguish trajectories that share the same spatial encoding, thereby reducing false positives during query processing. However, existing trajectory indexing schemes still face three key limitations.

Limitation 1: Spatial-Code Fragmentation. Existing SFC-based trajectory indexes construct their encoding space over the entire quadtree, despite highly uneven trajectory distributions. Empty cells leave unused code regions, while low-density cells insert only a few trajectory keys between densely populated regions. Both cases fragment the spatial codes of dominant trajectory groups. As shown in Fig. 1b, empty cells B and C separate the codes of cells A and D, while the low-density cell E, F interrupts those of the densely populated cells D and G. Such fragmentation produces numerous disjoint scan intervals, increasing range-localization and scan overhead.

Limitation 2: Static Spatial Encoding Order. Conventional SFCs assign spatial codes according to a predefined traversal order. When this order is inconsistent with the trajectory distribution or query-access pattern, cells frequently accessed together may receive non-consecutive codes. In Fig. 1b, the learned key space places cells D and G consecutively, preserving the code continuity of most trajectories in this region. Although existing studies [11, 21, 30] learn SFCs from historical query workloads. However, an order optimized primarily for a particular workload may lose its locality when the query distribution changes, resulting in performance degradation under unseen workloads.

Limitation 3: Fixed Shape-Encoding Granularity. Existing trajectory indexes typically use a single global enlarged-element partition for shape encoding. A finer partition improves shape discrimination but also enlarges the shape-code space and increases storage and filtering overhead. As illustrated in Fig. 1c, the trajectories in the left enlarged element are already distinguishable with a coarser partition, making the 3×3 partition unnecessarily fine-grained. In contrast, the trajectories on the right produce identical bitmaps under the 2×2 partition and cannot be distinguished during filtering. Thus, a fixed global granularity cannot simultaneously provide compact encoding and effective filtering across heterogeneous regions.

To address these limitations, we propose **LETI** (**L**earned Space-Filling Curves for **E**fficient **T**rajectory **I**ndexing), a trajectory indexing framework for KV stores that jointly optimizes the spatial hierarchy, encoding order, and shape granularity. To reduce spatial-code fragmentation, LETI constructs a compact quadtree [10] by collapsing low-density cells and reassigning their trajectories to retained ancestors. The resulting hierarchy better reflects the trajectory distribution and avoids sparse key-space regions that separate dense trajectory groups. To improve scan locality, LETI learns the encoding order of the retained cells using reinforcement learning. By considering both trajectory distribution and query workload, the learned order groups frequently co-accessed cells while preserving spatial locality, reducing fragmented scan intervals and sensitivity to workload shifts. Finally, LETI employs adaptive shape encoding within each enlarged element. It selects the partition granularity according to local trajectory characteristics, balancing shape discriminability against encoding and filtering overhead. In summary, we make the following contributions:

- We propose LETI, a trajectory indexing framework for KV stores that jointly optimizes the spatial hierarchy, encoding order, and shape granularity.
- We formulate encoding-order construction as a sequential decision problem and develop an RL-based method that jointly captures workload patterns and trajectory distributions, improving scan locality and robustness to workload shifts.
- We design an adaptive shape encoding scheme that selects the partition granularity according to local trajectory distributions, balancing filtering effectiveness and encoding overhead.
- We conduct extensive experiments on two real-world datasets, demonstrating LETI's query efficiency and robustness across different workloads.

Paper Organization. Section 2 introduces the preliminaries, followed by an overview of LETI in Section 3. Section 4 presents the proposed indexing method in detail, while Section 5 describes query processing and incremental index maintenance in LETI. Section 6 reports the experimental results. Related work is discussed in Section 7, and Section 8 concludes the paper. For ease of reference, Table 1 summarizes the main notations.

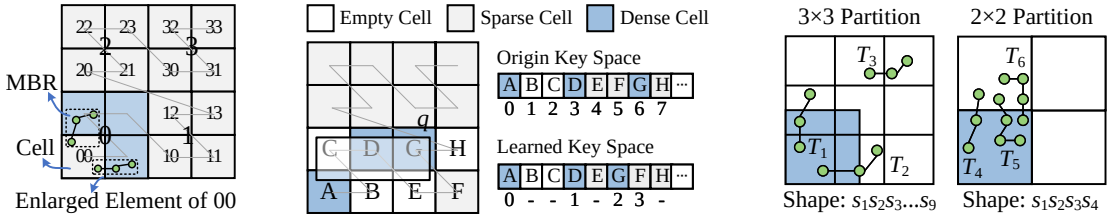


Fig. 1 SFC-based trajectory indexing and its limitations. (a) A trajectory is assigned the code of the minimum cell whose EE covers its MBR. (b) Empty and sparse cells may separate dense regions in key space, causing fragmented scans. (c) Fixed EE partitions may be overly fine-grained or overly coarse, leading to redundant encoding or weak filtering effectiveness.

2 Preliminaries

2.1 Definitions

Definition 1 (Trajectory). A trajectory is a time-ordered sequence of spatio-temporal points, denoted by $\mathcal{T} = \langle p_1, p_2, \ldots, p_m \rangle$, where $p_i = (x_i, y_i, t_i)$ records the spatial coordinates (x_i, y_i) of a moving object at timestamp t_i. The minimum bounding rectangle of $\mathcal{T}$, denoted by MBR($\mathcal{T}$), is the smallest axis-aligned rectangle that encloses all points in $\mathcal{T}$.

Definition 2 (Query Types). Three trajectory query types are considered: spatial range query, similarity search, and top-k query. Let $q = (x_{\min}, y_{\min}, x_{\max}, y_{\max})$ denote a spatial query window, $\mathbf{T} = \{\mathcal{T}_1, \ldots, \mathcal{T}_n\}$ a trajectory dataset, $\mathcal{T}_q$ a query trajectory, τ a similarity threshold, and $d(\cdot, \cdot)$ a trajectory distance function (e.g., Fréchet [1] or Dynamic Time Warping (DTW) [34]). The queries are defined as follows.

Spatial range query (SRQ). An SRQ returns all trajectories in $\mathbf{T}$ that intersect q:

$$\mathbf{T}_{\text{srq}} = \{\mathcal{T}_i \in \mathbf{T} \mid \mathcal{T}_i \cap q \neq \emptyset\}. \tag{1}$$

Similarity search (SS). Given $\mathcal{T}_q$ and τ, an SS returns all trajectories whose distance to $\mathcal{T}_q$ does not exceed τ:

$$\mathbf{T}_{\text{ss}} = \{\mathcal{T}_i \in \mathbf{T} \mid d(\mathcal{T}_q, \mathcal{T}_i) \leq \tau\}. \tag{2}$$

Top-k query. Given $\mathcal{T}_q$ and an integer k, a top-k query returns the k trajectories closest to $\mathcal{T}_q$:

$$\mathbf{T}_{\text{top-}k} = \arg \min_{\substack{\mathbf{S} \subseteq \mathbf{T} \\ |\mathbf{S}|=k}} \sum_{\mathcal{T}_i \in \mathbf{S}} d(\mathcal{T}_q, \mathcal{T}_i). \tag{3}$$

As discussed in TrajMesa [22,23], both SS and top-k queries take SRQ as a building block. As a result, we focus on SRQ for index design and query processing, while reporting the results for SS and top-k queries in Section 6.

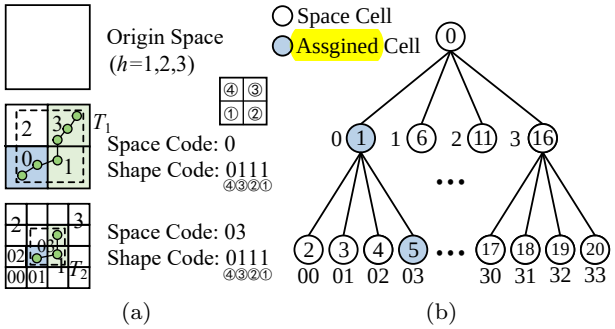


Fig. 2 Trajectory representation with spatial and shape encodings. (a) T_1 and T_2 are assigned to cells 0 and 03, respectively, yet share the same shape bitmap (0111) under a 2×2 enlarged-element partition. (b) Spatial cells form a quadtree, where node labels denote paths and integers denote XZ-order encodings; highlighted nodes correspond to the assigned cells.

Definition 3 (Space-Filling Curve, SFC). A space-filling curve is a mapping

$$\pi : \mathbb{R}^D \to \mathbb{R} \tag{4}$$

that transforms multi-dimensional spatial locations into a one-dimensional space while preserving spatial locality.

Definition 4 (Enlarged Element, EE). Given a cell $c = (x_{\min}, y_{\min}, x_{\max}, y_{\max})$, let $w_c = x_{\max} - x_{\min}$ and $h_c = y_{\max} - y_{\min}$ denote its width and height. The enlarged element of c is defined by doubling its width and height:

$$\text{EE}(c) = (x_{\min}, y_{\min}, x_{\min} + 2w_c, y_{\min} + 2h_c). \tag{5}$$

2.2 Trajectory Encoding

As shown in Fig. 2, to support spatial indexing in KV stores, the spatial domain is recursively partitioned by a quadtree up to a maximum depth h. Each cell c corresponds to a node in the hierarchy and is associated with 1) a quadtree path label indicating its position in the tree, and 2) a one-dimensional spatial encoding

$\pi(c)$ induced by the Z-order. For example, cell 0 has descendants 00, 01, 02, and 03, whose corresponding encodings are given by the integers inside the nodes.

To enable trajectory indexing, each trajectory is mapped to a one-dimensional key through a two-part representation consisting of a *spatial encoding* and a *shape encoding*.

Spatial Encoding. Given a trajectory $\mathcal{T}$, its assigned cell is determined as follows: 1) Consider all hierarchical cells that contain the lower-left corner of MBR($\mathcal{T}$). These cells form a chain from the root to a leaf in the quadtree. 2) Among these candidate cells, choose the deepest cell c such that its enlarged element EE(c) fully covers MBR($\mathcal{T}$). We denote the assigned cell of a trajectory $\mathcal{T}$ as $c_{\mathcal{T}}$, and $\pi(c_{\mathcal{T}})$ the integer encoding. In Fig. 2a, $\mathcal{T}_1$ is assigned to cell 0 at level 1 with $\pi(c_{\mathcal{T}_1}) = 1$, whereas $\mathcal{T}_2$ to cell 03 at level 2 with $\pi(c_{\mathcal{T}_2}) = 5$.

Shape Encoding. To capture the coverage of a trajectory within EE(c), the enlarged element is partitioned into an $n_x \times n_y$ grid, and the trajectory is represented as a bitmap $s(\mathcal{T}) \in \{0,1\}^{n_x \times n_y}$, where each bit indicates whether $\mathcal{T}$ intersects the corresponding subcell. In Fig. 2a, the shadow subcells are those intersected by the trajectory and are therefore set to 1; both $\mathcal{T}_1$ and $\mathcal{T}_2$ yield the same shape bitmap 0111.

A trajectory key combines the spatial encoding $\pi(c)$ and the shape encoding. Because the shape space under an $n_x \times n_y$ partition contains up to $2^{n_x \times n_y}$ bitmaps, directly encoding all shapes is inefficient. Following TShape [15], shape bitmaps are compacted locally within each cell. Let $\mathcal{T}(c)$ be the set of trajectories assigned to cell c. The set of distinct shapes in c is

$$\mathcal{S}(c) = \{s(\mathcal{T}) \mid \mathcal{T} \in \mathcal{T}(c)\}. \tag{6}$$

Each shape bitmap is remapped to a local shape code $s^*(\mathcal{T}) \in \{1, 2, \ldots, |\mathcal{S}(c)|\}$. The trajectory encoding is defined as

$$\text{TrajEncode}(\mathcal{T}) = (\pi(c_{\mathcal{T}}) \ll b) \mid s^*(\mathcal{T}), \tag{7}$$

where $\ll$ denotes the left shift operation and b is the number of bits reserved for shape codes. Let N be the maximum number of observed shapes in any cell:

$$N = \max_{c \in \mathcal{C}} |\mathcal{S}(c)|. \tag{8}$$

The bit width is set to $b = \lfloor \log_2 N \rfloor + 1$, providing sufficient code space for observed shapes while reserving space for newly emerging shapes and keeping the spatial encoding in the higher-order bits.

Given a trajectory $\mathcal{T}$, its key in the KV store is:

$$\text{key}(\mathcal{T}) = \mathit{shard} :: \text{TrajEncode}(\mathcal{T}) :: \mathit{tid}, \tag{9}$$

Table 1 Main notation used throughout the paper.

Symbol	Meaning
$\mathbf{T}$ (resp. $\mathcal{T}$)	trajectory dataset (resp. a trajectory)
q (resp. $\mathcal{W}$)	spatial query window (resp. a query workload)
h (resp. ℓ)	maximum quadtree depth (resp. a quadtree level)
$\mathcal{T}(c)$	trajectories assigned to cell c
MBR($\mathcal{T}$)	minimum bounding rectangle of $\mathcal{T}$
EE(c)	enlarged element of cell c
$\pi(c)$	one-dimensional encoding of cell c
$n_x \times n_y$	enlarged-element partition size
$s(\mathcal{T})$ (resp. $s^*(\mathcal{T})$)	shape bitmap (resp. compact local shape code)

where *shard* is a randomly assigned prefix used to alleviate hotspot caused by spatial locality, and *tid* uniquely identifies the trajectory. Following Trass [14], we use 8 shards, i.e., $\mathit{shard} \in \{0, 1, \ldots, 7\}$. Our focus is on TrajEncode($\mathcal{T}$), while the *shard* prefix and *tid* are used following the standard KV-store design.

2.3 Retrieval Workflow

Given a spatial range query q, query processing follows a filter-and-refine workflow that narrows the search space using both spatial and shape encodings. Let $\mathcal{C}_q$ denote the set of cells whose enlarged elements intersect q.

1. Spatial Filtering. For each $c \in \mathcal{C}_q$, we use its spatial encoding $\pi(c)$ to locate corresponding key regions. Each $\pi(c)$ is associated with a shape cache ShapeCache(c), which stores distinct shape tuples $\langle s, s^* \rangle$ in cell c, where s is the raw shape bitmap and s^* its compact local code.

2. Shape Filtering. For each $c \in \mathcal{C}_q$, construct a query shape signature $\sigma_q(c)$ over EE(c) using the same $n_x \times n_y$ partition as trajectory encoding. A bit is set to 1 if the corresponding subcell intersects q. The signature is matched against ShapeCache(c), and a tuple $\langle s, s^* \rangle$ is retained only if $\sigma_q(c) \wedge s \neq 0$.

3. Key Scan and Refinement. The retained local shape codes are combined with $\pi(c)$ and each *shard* to form candidate key prefixes (i.e., intervals containing a single key). These prefixes are scanned in the KV store to retrieve candidate trajectories, which are then verified by exact geometric intersection.

Limitations. The efficiency of this workflow depends on producing compact key intervals and effective filtering. If spatially related cells are scattered in the key order, a query may generate many fragmented key intervals, increasing scan overhead. Similarly, if trajectories share the same shape encoding, filtering becomes less

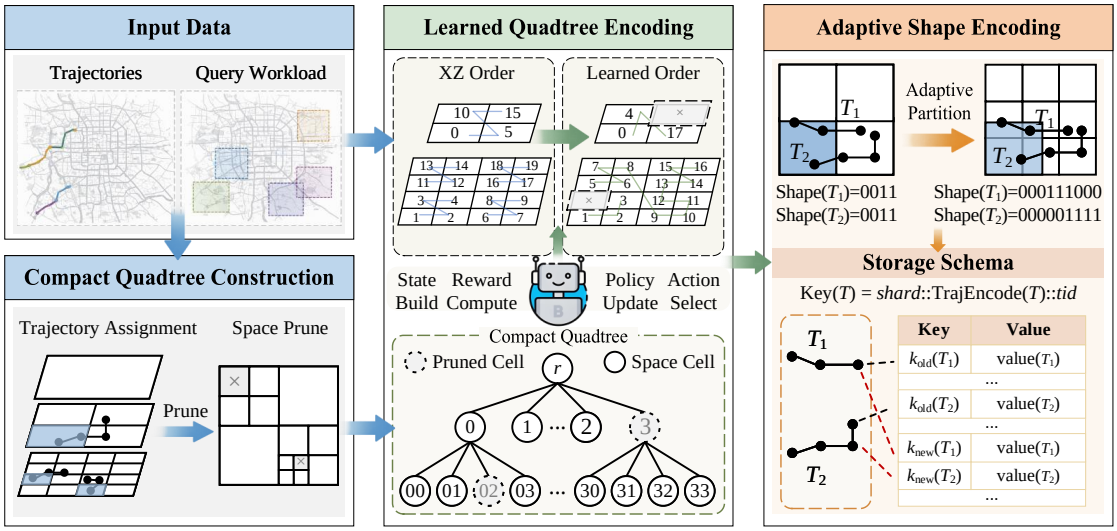


Fig. 3 Overview of the LETI framework.

selective and more candidates pass to refinement. Both issues increase scan and verification cost, motivating the design of LETI.

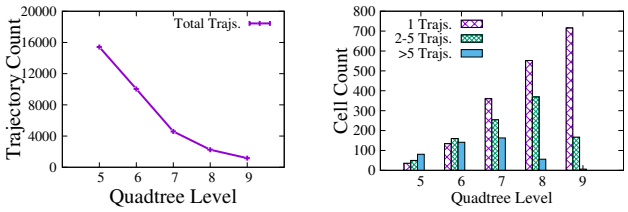
3 Overview of LETI

The overall framework of LETI is illustrated in Fig. 3. LETI improves trajectory indexing by jointly optimizing the spatial hierarchy, encoding order, and shape granularity. Together, these components produce compact and locality-preserving trajectory keys that reduce interval localization, scanning, and filtering overhead in KV stores. The framework consists of three stages.

1. Compact Quadtree Construction. This stage refines the spatial hierarchy. A quadtree is constructed according to the trajectory distribution, and low-density cells are collapsed into retained ancestors. The subsequent encoding is performed over the resulting compact structure.

2. Learned Quadtree Encoding. This stage determines the encoding order over the compact quadtree. Instead of using a predefined spatial order, LETI employs RL to learn a space-filling curve from both the trajectory distribution and query workload. The learned order groups frequently co-accessed cells while preserving spatial locality, reducing fragmented scan intervals and improving robustness to workload shifts.

3. Adaptive Shape Encoding. This stage refines the shape encoding within each enlarged element. LETI selects the partition granularity according to the local trajectory distribution. By balancing encoding overhead and shape discriminability, it improves the effectiveness of shape-based filtering.



(a) Trajectory distribution. (b) Cell distribution.

Fig. 4 Trajectory distribution across quadtree levels on T-Drive dataset.

4 Methodology

4.1 Compact Quadtree Construction

Conventional SFC-based indexes linearize a predefined quadtree without considering the trajectory distribution. Under a skewed distribution, empty or low-density cells may separate densely populated cells in the traversal order, fragmenting their trajectory keys across the key space.

Fig. 4 illustrates this imbalance on the T-Drive dataset. Trajectory assignments are concentrated at a few intermediate levels, particularly levels 5–6 (Fig. 4a), whereas most cells at levels 7 and deeper contain fewer than five trajectories (Fig. 4b). Retaining these sparse cells fragments the key space and weakens the locality of densely populated regions. To reduce this fragmentation, LETI constructs a compact quadtree adapted to the trajectory distribution, as illustrated in Fig. 5a.

Each retained cell represents a spatial region and is associated with three attributes: *qPath*, the root-to-cell path that identifies its spatial location; *qOrder*, its position in the learned traversal used to construct tra-

Algorithm 1 Bottom-up Quadtree Pruning

Input: Quadtree $\mathcal{Q}$, pruning threshold δ
Output: Compact quadtree $\mathcal{Q}'$
1: $\mathcal{P} \leftarrow \varnothing$
2: **for** each level ℓ from bottom to top **do**
3: **for** each non-root cell c at level ℓ **do**
4: **if** $|\mathcal{T}(c)| < \delta$ **then**
5: $p \leftarrow \text{parent}(c)$; $\mathcal{T}(p) \leftarrow \mathcal{T}(p) \cup \mathcal{T}(c)$
6: $\mathcal{T}(c) \leftarrow \varnothing$; $\mathcal{P} \leftarrow \mathcal{P} \cup \{c\}$
7: $\mathcal{Q}' \leftarrow \text{Contract}(\mathcal{Q}, \mathcal{P})$
8: **return** $\mathcal{Q}'$

jectory keys (Section 4.2); and *sIntervals*, the merged key intervals associated with the cell and its retained descendants, which enable direct range generation when the cell is fully covered by a query (Section 5.2).

To construct the compact quadtree, each trajectory $\mathcal{T}$ is assigned to a spatial cell c according to MBR($\mathcal{T}$). Let $\mathcal{T}(c)$ denote the trajectories assigned to c. Given a pruning threshold δ, LETI processes the quadtree bottom-up. If a non-root cell satisfies $|\mathcal{T}(c)| < \delta$, its trajectories are propagated to its parent and the cell is marked for removal. After pruning, each retained cell is connected to its nearest retained ancestor.

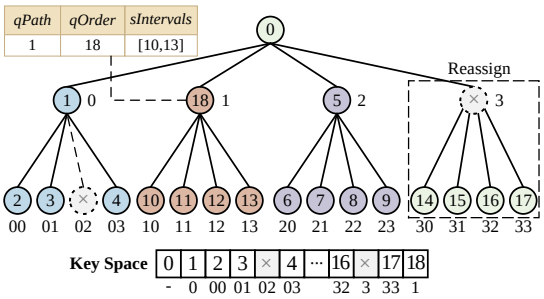
Algorithm 1 summarizes this procedure. LETI scans the quadtree bottom-up, propagating trajectories from each low-density cell to its parent and marking the cell for removal (Lines 4–6). After all levels are processed, the marked cells are removed and each retained cell is connected to its nearest retained ancestor (Line 10), producing the compact quadtree.

Let N denote the number of cells, M the total number of trajectory assignments, and h the maximum depth. The bottom-up traversal and final tree contraction each take $O(N)$ time. A trajectory may be propagated through at most h ancestors, yielding $O(Mh)$ reassignment time. Therefore, the pruning procedure takes $O(N + Mh)$ time. The quadtree, trajectory assignments, and pruning markers require $O(N + M)$ space.

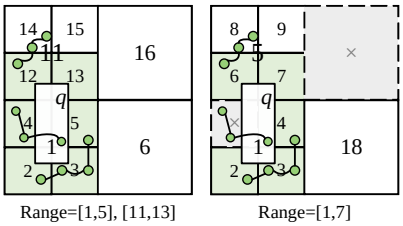
4.2 Learned Quadtree Encoding

While the compact quadtree removes structural redundancy, it does not determine how retained cells are mapped into one-dimensional keys. In KV stores, the traversal order directly affects the compactness of generated key intervals. If related cells are placed far apart in the key space, a single spatial query may result in multiple fragmented scans, increasing both index lookup and data access cost.

Fixed space-filling curves such as XZ-order [5] preserve geometric locality through a predefined traversal but do not account for the trajectory distribution or



(a) Compact quadtree



(b) XZ-order (c) Learned order

Fig. 5 Effect of traversal order on key-interval generation.

query workload. Workload-driven approaches [11, 21, 30] optimize the encoding order for historical queries, but their locality may degrade when the workload changes. LETI instead learns an encoding order that considers both trajectory distribution and query workload. The trajectory distribution captures workload-independent spatial relationships, while the query workload identifies cells that are frequently accessed together.

For trajectory indexing, the traversal order should reflect both *geometric proximity* and *trajectory distribution* while accounting for cells that are frequently accessed together. The compact quadtree exposes both spatial structure and trajectory distribution, providing a natural basis for constructing such a traversal.

Thus, LETI learns a traversal order directly over the compact quadtree. As illustrated in Fig. 5a, the traversal is defined after low-density nodes are pruned, with the crossed-out nodes (e.g., 02 and 3) excluded from the ordering. Figures 5b and 5c illustrate the advantage of the learned layout over a fixed SFC for query processing. Under XZ-order, cells intersecting the same query window q are mapped to disjoint positions in the key space. Consequently, the query is decomposed into multiple intervals, such as $[1, 5]$ and $[11, 13]$. In contrast, the learned traversal places these cells consecutively, allowing the same query to be covered by a single interval $[1, 7]$. This reduces the number of generated scan ranges and improves scan locality.

MDP Formulation. Let $\mathcal{C} = \{c_1, \ldots, c_N\}$ denote the set of cells in the compact quadtree, and let $\mathcal{W} = \{q_1, \ldots, q_K\}$ denote a query workload. Each cell c_i is

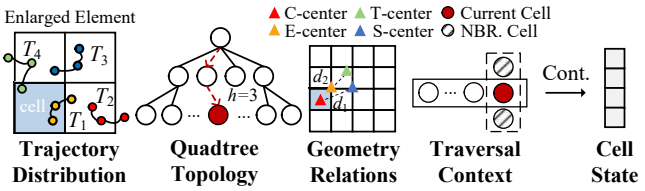


Fig. 6 State representation of a cell.

associated with a trajectory set $\mathcal{T}(c_i)$. We aim to learn a bijective mapping

$$\pi : \mathcal{C} \to \{1, 2, \ldots, |\mathcal{C}|\}, \tag{10}$$

which defines a traversal order over the cells. The objective is to place cells that are spatially close and distributionally similar at nearby positions in the one-dimensional key space, so that queries from $\mathcal{W}$ can be evaluated with fewer and more compact key intervals.

Finding an optimal mapping π is a combinatorial optimization problem with a search space of size $|\mathcal{C}| \times (|\mathcal{C}|-1) \cdots \times 1$. Rather than searching this space directly, we observe that a traversal order can be constructed incrementally: at each step, a cell is selected from the remaining candidates and appended to the current sequence. Each decision influences subsequent choices and the quality of the final traversal. We therefore formulate traversal construction as a Markov Decision Process (MDP) and learn a data-aware space-filling curve using reinforcement learning.

State Representation. At decision step t, a candidate cell c_t is represented by $s_t = \phi(c_t, \mathcal{T}(c_t), \pi_t)$, where $\mathcal{T}(c_t)$ denotes the trajectories associated with c_t, and π_t is the partial traversal constructed so far. As illustrated in Fig. 6, the state combines four types of features.

1. Trajectory Distribution. Considering that cells sharing more trajectories are likely to exhibit stronger spatial locality, the first component characterizes trajectories around c_t based on their spatial relations with the cell and its enlarged element $\mathrm{EE}(c_t)$. As shown in Fig. 6, trajectories are grouped into four categories: (i) intersecting c_t and covered by $\mathrm{EE}(c_t)$ (e.g., $\mathcal{T}_1$), (ii) intersecting $\mathrm{EE}(c_t)$ but not c_t (e.g., $\mathcal{T}_2$), (iii) covered by $\mathrm{EE}(c_t)$ but not intersecting c_t (e.g., $\mathcal{T}_3$), and (iv) intersecting c_t but not covered by $\mathrm{EE}(c_t)$ (e.g., $\mathcal{T}_4$). Their normalized proportions form $\mathbf{f}_{\mathrm{traj}}(c_t)$, capturing local density and distribution patterns.

2. Quadtree Topology. The second component encodes the position of c_t in the compact quadtree, including its level and path from the root. These features form $\mathbf{f}_{\mathrm{quad}}(c_t)$, providing hierarchical context beyond spatial coordinates.

3. Geometric Relation. It captures relative spatial relations using representative centers within the EE, including the cell center (C-center), EE center (E-center), trajectory centroid of assigned trajectories (T-center), and global centroid (S-center). Pairwise distances among these centers, e.g., d_1 and d_2 in Fig. 6, form $\mathbf{f}_{\mathrm{geo}}(c_t)$, capturing geometric proximity and distributional affinity.

4. Traversal Context. The final component captures dynamic information from the current traversal, including the visited ratio $|\pi_t|/|\mathcal{C}|$, the distance to the previously selected cell, and density statistics of neighboring cells. These features form $\mathbf{f}_{\mathrm{ctx}}(c_t)$, guiding the next transition.

The final state representation concatenates all components:

$$s_t = [\mathbf{f}_{\mathrm{traj}}(c_t), \mathbf{f}_{\mathrm{quad}}(c_t), \mathbf{f}_{\mathrm{geo}}(c_t), \mathbf{f}_{\mathrm{ctx}}(c_t)]\,. \tag{11}$$

Action Space and Transition. At step t, let c_t be the current cell and $\mathcal{V}_t \subseteq \mathcal{C}$ the set of visited cells. An action selects the next cell c_{t+1} from the unvisited set $\mathcal{C} \setminus \mathcal{V}_t$. Allowing transitions to any unvisited cell results in up to $O(|\mathcal{C}|)$ candidate actions and may introduce long jumps in the traversal. LETI therefore restricts the action space to a local neighborhood of c_t:

$$\mathcal{A}(c_t) = \mathcal{N}_{\mathrm{sp}}(c_t) \cup \mathcal{N}_{\mathrm{hier}}(c_t), \tag{12}$$

where $\mathcal{N}_{\mathrm{sp}}(c_t)$ contains spatially adjacent cells at the same quadtree level, and $\mathcal{N}_{\mathrm{hier}}(c_t)$ contains the parent and retained children of c_t in the compact quadtree. The valid action set is

$$\widetilde{\mathcal{A}}_t = \mathcal{A}(c_t) \cap (\mathcal{C} \setminus \mathcal{V}_t). \tag{13}$$

When $\widetilde{\mathcal{A}}_t$ is nonempty, the policy selects c_{t+1} from this set. If no local candidate remains, LETI moves to the nearest unvisited cell:

$$c_{t+1} = \arg\min_{c \in \mathcal{C} \setminus \mathcal{V}_t} d_{\mathrm{ctr}}(c_t, c), \tag{14}$$

where d_{ctr} denotes the distance between cell centers. This fallback ensures that every retained cell is visited while limiting discontinuities in the traversal.

After selecting c_{t+1}, LETI appends it to the partial traversal and updates

$$\pi_{t+1} = \pi_t \parallel c_{t+1}, \qquad \mathcal{V}_{t+1} = \mathcal{V}_t \cup \{c_{t+1}\}. \tag{15}$$

The next state s_{t+1} is then constructed from the updated traversal.

Reward Design. The reward evaluates both the local continuity of consecutive cells and the query cost of the resulting traversal. LETI therefore combines dense local feedback with a global reward periodically evaluated on the historical query workload.

1. Local Reward. In early stages of sequence construction, the global traversal quality is not yet observable. We therefore provide dense local feedback to guide the policy toward transitions that preserve spatial and distributional consistency.

Given consecutive cells c_t and c_{t+1}, we define a proximity reward based on their normalized center distance d_{ctr}:

$$\text{Rew}_{\text{prox}}(c_t, c_{t+1}) = 1 - \frac{d_{\text{ctr}}(c_t, c_{t+1})}{d_{\text{ctr}}^{\max}}. \tag{16}$$

Let $\mathcal{T}_\cap(c)$ denote the set of trajectories that intersect cell c. To capture the distributional affinity between consecutive cells, we define a similarity reward as the Jaccard similarity [17] between their intersecting trajectory sets:

$$\text{Rew}_{\text{sim}}(c_t, c_{t+1}) = \frac{|\mathcal{T}_\cap(c_t) \cap \mathcal{T}_\cap(c_{t+1})|}{|\mathcal{T}_\cap(c_t) \cup \mathcal{T}_\cap(c_{t+1})|}. \tag{17}$$

Treating spatial proximity and trajectory similarity equally, the local reward is defined as

$$\text{Rew}_{\text{local}} = \frac{\text{Rew}_{\text{prox}} + \text{Rew}_{\text{sim}}}{2}. \tag{18}$$

2. Global Reward. Local coherence alone does not ensure a globally effective traversal. We therefore evaluate the current traversal periodically using a global reward derived from the historical query workload $\mathcal{W}$. At each evaluation, the remaining unvisited cells are appended in XZ-order to form a complete traversal, which is then used for cost estimation.

In a KV store, query efficiency is largely determined by the scan ranges generated after mapping a query window to one-dimensional keys. A desirable traversal should produce few scan ranges and, when multiple ranges are unavoidable, distribute them regularly in the key space. Inspired by the cohesion measures introduced in prior work [30], we define a cohesion cost that captures both range fragmentation and gap regularity.

For a query q, suppose traversal π produces m scan ranges $[l_1, r_1], [l_2, r_2], \ldots, [l_m, r_m]$, where $r_i < l_{i+1}$. The length of the i-th gap and its normalized proportion are defined as

$$n_i = l_{i+1} - r_i, \qquad \mu_i = \frac{n_i}{\sum_{j=1}^{m-1} n_j}. \tag{19}$$

For $m \geq 3$, we measure gap regularity using normalized entropy:

$$H_{\text{gap}}(q, \pi) = -\frac{1}{\log(m-1)} \sum_{i=1}^{m-1} \mu_i \log \mu_i. \tag{20}$$

When $m \leq 2$, we define $H_{\text{gap}}(q, \pi) = 1$, since there is no variation among the gaps.

For example, consider two traversals that both produce three scan ranges with the same total gap length. The first produces $[1, 3]$, $[6, 8]$, and $[11, 13]$, with gap lengths $(3, 3)$. The second produces $[1, 3]$, $[5, 7]$, and $[11, 13]$, with gap lengths $(2, 4)$. The former has higher normalized gap entropy and therefore exhibits more regular spacing.

The cohesion cost of query q under traversal π is

$$C_{\text{coh}}(q, \pi) = m + 1 - H_{\text{gap}}(q, \pi). \tag{21}$$

The number of ranges m captures scan fragmentation, while $1 - H_{\text{gap}}(q, \pi)$ penalizes irregular gap distributions. Consequently, a lower cost indicates fewer and more regularly distributed scan ranges. The cohesion cost over workload $\mathcal{W}$ is the average cost of its queries:

$$C_{\text{coh}}(\mathcal{W}, \pi) = \frac{1}{|\mathcal{W}|} \sum_{q \in \mathcal{W}} C_{\text{coh}}(q, \pi). \tag{22}$$

Let π_z denote the baseline XZ-order. The global reward is defined as the relative reduction in cohesion cost:

$$\text{Rew}_{\text{global}} = \frac{C_{\text{coh}}(\mathcal{W}, \pi_z) - C_{\text{coh}}(\mathcal{W}, \pi)}{C_{\text{coh}}(\mathcal{W}, \pi_z)}. \tag{23}$$

A positive reward indicates that the learned traversal produces a lower cohesion cost than XZ-order.

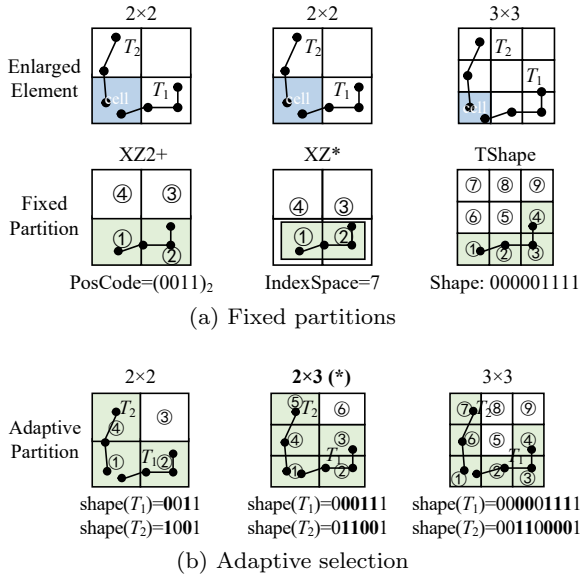
Policy Learning. LETI trains the traversal policy using an Actor–Critic framework [19]. Given state s_t, the Actor scores the candidate cells. A state-dependent mask excludes non-neighboring and previously visited cells, and the remaining scores are normalized over the valid action set $\widetilde{\mathcal{A}}_t$. The Critic estimates the state value v_t, which serves as a baseline for variance reduction. Both networks contain two hidden layers with 256 units per layer and use ReLU activations.

At each step, LETI selects the next cell from the masked policy, updates the partial traversal, and receives the local reward. The global reward is added when the traversal is periodically evaluated on the historical workload.

Once a complete traversal has been constructed, LETI computes advantage estimates using Generalized Advantage Estimation (GAE) [35]. The Actor and Critic are then updated using Proximal Policy Optimization (PPO) [36]. We train both networks for at most 400 episodes using a learning rate of 3×10^{-4} and a PPO clipping ratio of 0.2. The policy is evaluated every 40 episodes.

4.3 Adaptive Shape Encoding

Quadtree-based spatial encoding captures only coarse trajectory location. To support finer pruning, prior



(a) Fixed partitions

(b) Adaptive selection

Fig. 7 Adaptive shape encoding with fixed vs. adaptive partitions.

work [14, 15, 23] augments each spatial encoding with a shape encoding over the corresponding enlarged element using fixed partitioning schemes. As shown in Fig. 7a, XZ2+ [23] and XZ* [14] use a fixed 2×2 EE and a matching 2×2 partition. XZ2+ encodes coverage as a bitmap, while XZ* maps coverage patterns to compact integer identifiers. TShape [15] follows the same design principle, but supports larger EE and finer partitions. Although these designs differ in encoding format, they apply one partitioning rule to all trajectories within the same EE. This coupling limits shape filtering under heterogeneous trajectory distributions. Coarse partitions may provide insufficient separation, whereas finer partitions enlarge the shape domain exponentially (up to $2^{n_x \times n_y}$), increasing encoding space and index size without proportional pruning benefit.

Rather than fixing one partition per EE type, LETI selects a partition for each EE based on its assigned trajectories, aiming to use as few bits while preserving shape discriminability. Fig. 7b illustrates this trade-off. For the same EE, a 2×2 partition distinguishes trajectories $\mathcal{T}_1$ and $\mathcal{T}_2$ in only two bits. A 3×3 partition increases this to five bits, but requires a 9-bit encoding. A 2×3 partition yields four differing bits using only 6 bits, while avoiding weakly informative or empty subcells. It thus provides stronger discrimination than 2×2, while using fewer bits and higher spatial utilization than 3×3.

To quantify this trade-off, we evaluate each candidate partition using two metrics: 1) an *occupancy ratio* $\rho(c, n_x, n_y)$ that measures utilization of the $n_x \times n_y$ grid, and 2) a *discriminability* score $\kappa(c, n_x, n_y)$ that measures separability of trajectory shapes. We combine them into a utility score

$$\text{score}(c, n_x, n_y) = \rho(c, n_x, n_y) \times \kappa(c, n_x, n_y). \tag{24}$$

A larger score indicates better partition utilization and stronger shape separation. The occupancy ratio is defined as

$$\rho(c, n_x, n_y) = \frac{|c_{\text{cross}}|}{n_x \times n_y}, \tag{25}$$

where $|c_{\text{cross}}|$ is the number of subcells intersected by at least one trajectory.

Let $\widehat{\mathcal{T}}(c)$ denote a sample of trajectories assigned to cell c, and let $M = \binom{|\widehat{\mathcal{T}}(c)|}{2}$. We define discriminability as the average normalized Hamming distance [13]:

$$\kappa(c, n_x, n_y) = \frac{1}{M} \sum_{\mathcal{T}_i, \mathcal{T}_j \in \widehat{\mathcal{T}}(c)} \frac{d_{\text{ham}}(s(\mathcal{T}_i), s(\mathcal{T}_j))}{n_x n_y}. \tag{26}$$

Given these metrics, LETI determines the partition via a grid search over candidate grid sizes. Specifically, we enumerate (n_x, n_y) within a bounded range and select the configuration that maximizes the utility score $\text{score}(c, n_x, n_y)$. In practice, we set $n_x, n_y \in [2, 8]$. This range ensures that the resulting bitmap length does not exceed $n_x \times n_y \leq 64$ bits (i.e., 8 bytes), while still providing sufficient flexibility to adapt to local trajectory distributions. Let $S \leq 64$ be the number of sampled trajectories and P the number of candidate configurations. Evaluating all candidates requires $O(P(SL + S^2))$ time, where L is the average number of sampled trajectory points processed when constructing a signature. With the fixed sampling and bitmap-size bounds used in LETI, the search cost is linear in P.

5 Query Processing and Maintenance

LETI supports query processing and incremental batch insertion over the same key layout. Both operations use the shape cache maintained for each retained cell. This section first introduces the shared cache, then presents spatial range query processing, followed by incremental insertion and index maintenance. LETI also supports similarity search and top-k queries under the same filter-and-refine framework by replacing geometric refinement with distance evaluation [22, 23].

5.1 Shape Cache

For each retained cell c, LETI maintains a shape cache $\text{ShapeCache}(c)$ shared by query processing and incremental insertion. It contains the distinct shape tuples $\langle s, s^* \rangle$ observed in c, where s is the shape bitmap and s^*

is its local shape code. During query processing, LETI tests each bitmap s against the query signature and combines the matching s^* with $\pi(c)$ to generate candidate scan keys. During insertion, an existing tuple is reused, while an unseen bitmap is assigned a new local shape code.

5.2 Spatial Range Query Processing

Given a query window q, LETI evaluates an SRQ through three stages: cell localization, candidate scan range generation, and exact refinement.

Cell Localization. LETI traverses the compact quadtree in breadth-first-search (BFS) order. For each visited retained cell c, let $E_c = \mathrm{EE}(c)$. Cells satisfying $E_c \cap q \neq \varnothing$ are identified as candidate cells for further processing.

Candidate Scan Range Generation. For each candidate cell c, LETI distinguishes two cases.

Case 1: Full coverage. If $E_c \subseteq q$, all trajectories associated with c and its retained descendants are candidates. LETI directly adds the precomputed subtree intervals $\mathrm{sIntervals}(c)$ as scan ranges without descending into the subtree.

Case 2: Partial intersection. If $E_c \cap q \neq \varnothing$ but $E_c \not\subseteq q$, LETI applies shape-based filtering in two steps. First, it computes a query signature $\sigma_q(c)$ over $\mathrm{EE}(c)$ and retains only the cached shape tuples $\langle s, s^* \rangle$ that satisfy $\sigma_q(c) \wedge s \neq 0$. Each retained tuple then yields a trajectory key and a singleton scan interval:

$$k_c = (\pi(c) \ll b) \,|\, s^*, \qquad I_c = [k_c, k_c + 1).$$

After processing both cases, LETI combines the resulting intervals and merges adjacent ones before issuing KV scans.

Algorithm 2 Candidate Scan Range Generation

Input: Query window q, compact-quadtree root r
Output: Candidate scan ranges $\mathcal{R}$
1: $\mathcal{L}_{\mathrm{bfs}} \leftarrow \{r\}$; $\mathcal{R} \leftarrow \varnothing$
2: **while** $\mathcal{L}_{\mathrm{bfs}} \neq \varnothing$ **do**
3: $c \leftarrow \mathrm{pop}(\mathcal{L}_{\mathrm{bfs}})$; $E_c \leftarrow \mathrm{EE}(c)$
4: **if** $E_c \subseteq q$ **then**
5: $\mathcal{R} \leftarrow \mathcal{R} \cup \mathrm{sIntervals}(c)$
6: **else if** $E_c \cap q \neq \varnothing$ **then**
7: compute the query signature $\sigma_q(c)$
8: **for** each $\langle s, s^* \rangle \in \mathrm{ShapeCache}(c)$ **do**
9: **if** $\sigma_q(c) \wedge s \neq 0$ **then**
10: $k_c \leftarrow (\pi(c) \ll b) \,|\, s^*$
11: $\mathcal{R} \leftarrow \mathcal{R} \cup \{[k_c, k_c + 1)\}$
12: $\mathcal{L}_{\mathrm{bfs}} \leftarrow \mathcal{L}_{\mathrm{bfs}} \cup \mathrm{children}(c)$
13: $\mathcal{R} \leftarrow \mathrm{MergeAdjacent}(\mathcal{R})$
14: **return** $\mathcal{R}$

Algorithm 2 summarizes candidate range generation. Starting from the compact-quadtree root, LETI visits cells in BFS order (Lines 2–12). For each cell, it obtains the corresponding enlarged element (Line 3). A fully covered cell contributes its precomputed subtree intervals (Lines 4–5), whereas a partially intersecting cell undergoes shape-based filtering (Lines 6–9). Each matching shape produces a singleton interval (Lines 10–11), and the cell's children are added for further examination (Line 12). Finally, overlapping or adjacent ranges are merged and returned (Lines 13–14).

Complexity. Let N_q be the number of visited cells and $\mathcal{C}_q^{\mathrm{part}}$ the set of partially intersecting cells. The total number of examined shape tuples is

$$M_q = \sum_{c \in \mathcal{C}_q^{\mathrm{part}}} |\mathrm{ShapeCache}(c)|.$$

Because each bitmap occupies at most 64 bits, traversal and filtering take $O(N_q + M_q)$ time. If K_q intervals are generated before merging, sorting and merging them takes $O(K_q \log K_q)$ time. Thus, range generation takes $O(N_q + M_q + K_q \log K_q)$ time and uses $O(N_q + K_q)$ working space. The subsequent KV scan and refinement costs depend on the numbers of scanned rows and retrieved trajectories.

KV Scan and Refinement. After candidate scan ranges are generated, LETI directly issues range scans to the KV store and retrieves candidate trajectories. Exact geometric intersection checks are then applied to remove false positives and produce final query results.

5.3 Incremental Insertion and Maintenance

LETI supports incremental batch insertion without immediately reconstructing the compact quadtree or retraining the learned traversal. For a new trajectory $\mathcal{T}$, LETI first determines its spatial cell using the same assignment process as index construction. If the cell has been pruned, $\mathcal{T}$ is assigned to its nearest retained ancestor c. LETI then computes its shape bitmap $s(\mathcal{T})$ using the partition of $\mathrm{EE}(c)$.

If $s(\mathcal{T})$ already occurs in $\mathrm{ShapeCache}(c)$, LETI reuses its local shape code $s^*(\mathcal{T})$. Otherwise, LETI assigns the next available local shape code and adds the new tuple $\langle s, s^* \rangle$ to the cache, provided that the code fits within the b reserved bits. LETI then generates the trajectory key and writes $\mathcal{T}$ to the primary table. Since the spatial layout remains unchanged, the qPaths, qOrders, and subtree intervals of the compact quadtree need not be recomputed.

As insertions accumulate, LETI tracks the fraction of newly observed shapes, shape-code space utilization,

and query performance. A large number of new shapes or noticeable degradation may require shape-code reorganization, while insufficient reserved space requires increasing b. Substantial changes in the spatial or workload distribution may require reconstruction of the compact quadtree and learned traversal. Because these operations change existing keys, they are performed offline together with row-key migration. LETI targets periodically updated trajectory repositories, where newly arriving trajectories are incorporated in batches between query phases. Concurrent ingestion and in-place trajectory modification are beyond the scope of this work.

6 Experiments

6.1 Experimental Settings

Datasets. We evaluate LETI on two real-world trajectory datasets and their scaled variants. All trajectories are pre-segmented before indexing following prior studies [25, 26, 38].

- **CD-Taxi** [37]. Contains 48,223 trajectories from 2,940 taxis in Chengdu, China, collected over 10 days (Oct 1–10, 2018) with an average sampling interval of 3.13s, covering 30.65–30.73°N and 104.04–104.13°E.
- **T-Drive** [44]. Contains GPS records from 10,357 taxis in Beijing (Feb 2–8, 2008), with about 15M points and an average sampling interval of 177s. We extract 318,774 trajectories, covering 39.6–40.1°N and 116.0–116.8°E.
- **T-Drive-Large**. Constructed for scalability evaluation using ×100, ×500, ×1000, and ×2000 variants of the original 530 MB dataset (up to 1.04 TB). To avoid duplication, samples are generated via data augmentation, including random perturbation, shape-preserving Gaussian noise, interpolation, random rotation/scaling, and feature-level mixing.

Query Workloads. We evaluate LETI using spatial range queries (SRQ) as the primary workload, along with similarity search and top-k queries for completeness.

For SRQ, we generate query workloads under three spatial distributions: *uniform*, *skewed*, and *Gaussian*. We partition each dataset into a 64×64 density grid. The *uniform* workload samples occupied cells with equal probability, the *Gaussian* workload is centered at the densest region, and the *skewed* workload samples eight spatially separated hotspots with Zipf probabilities. For every sampled center, we generate five nested square windows from $100 \times 100\,\mathrm{m}^2$ to $2000 \times 2000\,\mathrm{m}^2$. Each distribution has disjoint training, validation, warm-up, and test sets. Learned layouts are trained with 300 queries balanced across the five ranges, while each reported setting uses 100 warm-up and 100 test queries.

Similarity and top-k queries are constructed following the *skewed* distribution by sampling 100 query trajectories from the T-Drive dataset. Their evaluation follows the standard filter-and-refine process in [14].

Table 2 Experimental settings.

Parameter	Values
Maximum quadtree depth h	6, 7, **8**, 9, 10
Pruning threshold δ	2, **4**, 6, 8, 10
Workload distribution $\mathcal{D}$	Uniform, **Skewed**, Gaussian
Query window size r (m)	100, 500, **1000**, 1500, 2000

Baselines. We compare LETI with representative trajectory encodings based on XZ-order and learned SFC methods.

- **XZ*** [14]. Extends XZ-order by associating each trajectory with an EE partitioned into a fixed 2×2 grid, where the occupancy pattern serves as a coarse shape descriptor.
- **TShape** [15]. Generalizes XZ* by using a larger EE and finer partitioning, and maps shape bitmaps to compact integer codes to improve filtering granularity (we use a 3×3 setting in our implementation).
- **LMSFC** [11]. Learns a data-aware SFC via Sequential Model-Based Optimization (SMBO) [28], aiming to improve spatial locality for range queries.
- **BMTree** [21]. Learns a data-aware SFC traversal using Monte Carlo Tree Search (MCTS) [6], with the objective of optimizing spatial locality under given workloads.

Metrics and Parameter Settings. We use average query latency as the primary end-to-end metric. To interpret performance differences, we also report the number of retained cells (RC), row-key intervals (RKI), and candidate trajectories (CT). Training time is included in parameter studies to measure the cost of learning traversal orders.

We evaluate the impact of both index and workload parameters. The quadtree depth h controls spatial granularity, and the pruning threshold δ determines whether a cell is retained. On the workload side, the query distribution $\mathcal{D}$ defines the spatial access pattern, while the window size r controls the query range. Table 2 summarizes all evaluated settings, with default values highlighted in bold.

Implementation Details. All methods are implemented within the TMan [15] system to ensure a fair and unified comparison. Since LMSFC and BMTree are designed for point data, they are adapted to trajectories via SFC monotonicity [11]. Each trajectory is

Table 3 Query latency (ms) under different workloads and query ranges.

Dataset	Method	Skewed					Gaussian					Uniform				
		100	500	1000	1500	2000	100	500	1000	1500	2000	100	500	1000	1500	2000
T-Drive	LETI	**23.2**	**25.7**	**29.2**	**33.5**	**37.2**	**5.7**	**8.9**	**7.0**	**7.7**	**8.8**	**5.3**	**5.7**	**6.0**	**6.5**	**7.5**
	XZ*	45.0	39.7	42.5	45.4	48.3	16.0	16.1	18.1	19.6	20.9	10.6	10.7	11.6	12.0	13.1
	TShape	33.8	33.3	36.2	39.1	43.2	9.7	10.5	11.9	13.3	14.1	7.4	7.9	8.0	8.6	9.8
	LMSFC	51.0	52.2	57.7	57.7	61.3	29.6	31.3	33.2	33.8	35.6	48.0	48.1	48.3	49.6	51.3
	BMTree	53.9	56.5	59.9	62.8	66.3	50.5	51.2	53.0	54.5	55.0	74.9	75.5	79.8	84.0	87.2
CD-Taxi	LETI	**34.9**	**55.6**	**76.0**	**99.7**	**121.6**	**34.1**	**55.5**	**79.9**	**105.7**	**125.0**	**35.9**	**52.2**	81.4	**82.1**	**97.2**
	XZ*	99.8	124.9	129.1	145.9	164.1	93.7	109.8	133.6	164.2	185.6	65.5	77.7	93.9	115.6	127.5
	TShape	64.3	83.7	101.1	121.8	144.1	63.2	82.7	111.2	139.0	163.4	49.2	63.6	**78.9**	98.6	105.1
	LMSFC	71.6	88.1	104.3	122.3	142.4	57.8	73.8	95.1	113.7	131.4	107.4	118.4	131.4	144.1	156.9
	BMTree	79.1	96.5	112.4	132.3	155.1	64.9	91.4	118.2	137.9	157.6	100.5	106.5	122.1	138.5	152.4

mapped to a 1D interval using the SFC codes of the lower-left and upper-right corners of its MBR, enabling interval-based filtering. The same geometric refinement procedure is applied across all methods. For LETI, LMSFC, and BMTree, a traversal order is trained for each combination of workload distribution $\mathcal{D}$, depth h, and pruning threshold δ using the full dataset, and then applied during query processing.

All experiments are conducted on a single machine unless otherwise specified. Distributed experiments run on an eight-node cluster. We use HBase [2] as the KV store. Each node is equipped with an Intel Xeon E-2224G CPU (4 cores, 3.50 GHz), 128 GB RAM, and two 3.6 TB local disks.

6.2 Query Performance

We evaluate the average query latency of LETI and all baselines under different workload distributions and query ranges. As shown in Table 3, latency generally increases with the query range because larger windows require more range scans and retrieve more candidates for refinement.

LETI achieves the lowest latency in all 15 settings on T-Drive and in 14 of the 15 settings on CD-Taxi. TShape consistently ranks second on T-Drive, where LETI reduces its latency by 13.9%–42.1%. LETI also remains efficient as the query range expands. For example, on T-Drive, its latency increases by only 3.1 ms under the Gaussian workload and 2.2 ms under the uniform workload from 100 to 2000 m. This stability results from reducing both fragmented scan intervals and false positives.

On CD-Taxi under the Gaussian workload, LMSFC consistently outperforms the trajectory-specific TShape and XZ* indexes, reducing latency by 8.5%–19.6% and 28.8%–38.3%, respectively. This result demonstrates the value of combining a learned SFC with query partitioning under concentrated workloads. LMSFC also outperforms BMTree by 10.9%–19.5% because its query partitioning consolidates matched regions into fewer contiguous key intervals, reducing range localization and scan overhead.

6.3 Parameter Sensitivity

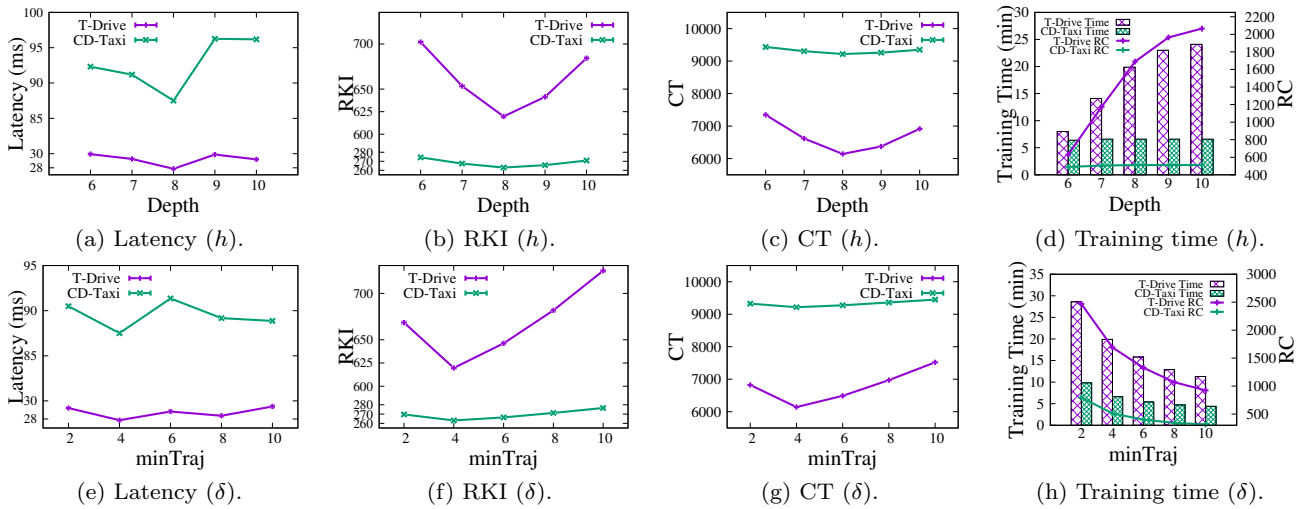
LETI has two main structural parameters: the maximum quadtree depth h and pruning threshold δ. The former controls spatial resolution, while the latter determines how aggressively low-density cells are removed. We evaluate their effects on query latency, RKI, CT, RC, and training time.

Effect of Depth h. Figs. 8a–8d show that query latency first decreases and then increases with h, reaching its minimum at $h = 8$ on both datasets. RKI and CT follow the same trend. A shallow quadtree cannot separate trajectories precisely, whereas an overly deep quadtree divides the space into too many cells and fragments the resulting key ranges.

A larger h also increases the learning cost. On T-Drive, RC grows from 631 to 2065 as h increases from 6 to 10, and training time rises from 8 to 24 min. On CD-Taxi, RC stabilizes at 511 from $h = 8$ onward, so training time remains approximately 7 min. We therefore use $h = 8$, which provides the lowest latency without unnecessarily enlarging the search space.

Effect of Threshold δ. Figs. 8e–8h show that a larger δ produces a smaller quadtree. From $\delta = 2$ to 10, RC decreases from 2470 to 924 on T-Drive and from 795 to 315 on CD-Taxi, reducing training time by more than half on both datasets.

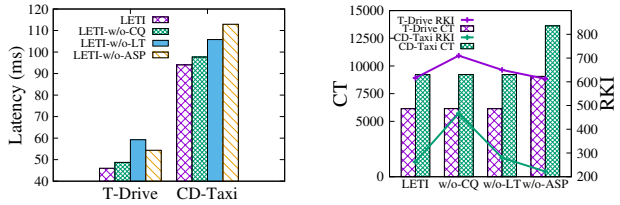
Query latency is lowest at $\delta = 4$. A smaller threshold retains too many sparse cells and produces more fragmented ranges, while a larger threshold assigns more trajectories to coarser cells and increases false positives. The corresponding changes in RKI and CT confirm this trade-off. We therefore set $\delta = 4$ by default.



(a) Latency (h). (b) RKI (h). (c) CT (h). (d) Training time (h).

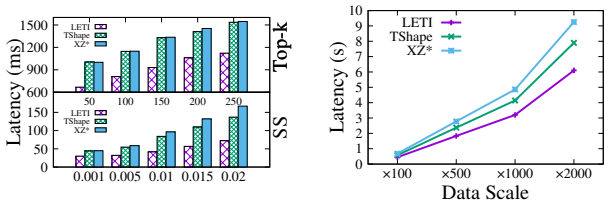
(e) Latency (δ). (f) RKI (δ). (g) CT (δ). (h) Training time (δ).

Fig. 8 Parameter sensitivity of LETI under varying quadtree depth h and pruning threshold δ.



(a) Latency. (b) CT and RKI.

Fig. 9 Ablation study of LETI.



(a) Similarity and top-k. (b) Scalability.

Fig. 10 Additional query types and distributed scalability of LETI.

6.4 Ablation Study

We compare LETI with three variants under the skewed workload with a query range of 1000 m. As shown in Fig. 9, removing any component increases query latency, but the source of the overhead differs.

CQ and LT mainly affect range localization. CQ removes low-density cells from the encoding space, and LT arranges the retained cells according to query workload and trajectory distribution. Without them, the generated key ranges become more fragmented. For example, RKI on T-Drive increases from 616 to 710 without CQ and to 650 without LT. The same pattern appears on CD-Taxi. Among the two components, LT has a larger effect on latency, especially on T-Drive, where removing it increases latency from 46 to 59 ms.

ASP mainly affects candidate filtering. Replacing adaptive partitions with the original fixed partition increases CT by about 48% on both datasets. A finer adaptive partition may produce slightly more RKI, but it distinguishes trajectory shapes more accurately and prevents many false positives from entering refinement. Without ASP, latency increases from 46 to 54 ms on T-Drive and from 94 to 113 ms on CD-Taxi.

6.5 Additional Query Types

We further evaluate LETI on similarity search and top-k queries over the skewed T-Drive workload. For similarity search, the Fréchet threshold ranges from 0.001 to 0.02, while for top-k queries, k varies from 50 to 250.

As shown in Fig. 10a, LETI consistently outperforms baselines. For similarity search, latency increases from 30 ms to 72 ms as the threshold grows, while TShape rises from 44 ms to 137 ms and XZ* from 45 ms to 168 ms, demonstrating effective pruning even with larger candidate sets.

A similar trend holds for top-k queries. As k increases from 50 to 250, LETI grows from 0.668 s to 1.122 s, compared with 1.004–1.535 s for TShape and 1.000–1.546 s for XZ*. The gap widens at larger k, where tighter scan ranges and more selective shape filtering eliminate candidates before ranking. The design of LETI enables it to remain effective for distance- and ranking-based workloads beyond spatial range queries.

6.6 Distributed Scalability

We evaluate LETI under increasing data sizes. Following Section 6.1, we construct four enlarged T-Drive variants and run all experiments on the same eight-node cluster, where $\times N$ denotes scaling by N. We compare LETI with TShape and XZ*. Fig. 10b reports average latency.

LETI consistently achieves the lowest latency. As the data size grows from $\times 100$ to $\times 2000$, latency increases from 0.5 s to 6.1 s for LETI, compared with 0.6–7.9 s for TShape and 0.7–9.3 s for XZ*. LETI grows more slowly, and the advantage becomes more pronounced at larger scales. In particular, the gap widens from 0.23 s at $\times 100$ to 3.15 s at $\times 2000$ against TShape.

This stems from jointly reducing scan overhead and refinement cost. The compact quadtree limits accessed cells, the learned traversal keeps co-accessed regions close in key space, and adaptive partitioning filters more false positives before refinement. As data volume grows, these effects accumulate and slow latency growth relative to baselines.

6.7 Query Distribution Shift

To evaluate the robustness of LETI under query distribution shift, we conduct experiments on T-Drive under the default settings. Traversal orders are trained on uniform, skewed, and Gaussian query distributions and evaluated on all three distributions. The skewed and Gaussian query sets are generated by perturbing a common uniformly distributed query set, so that the workloads differ only in their query distributions.

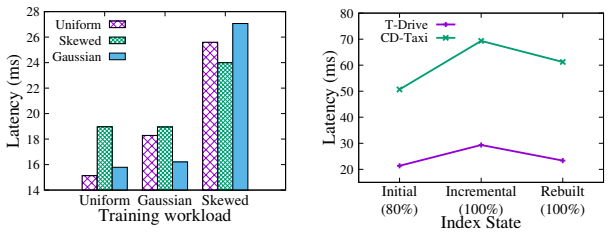
As shown in Fig. 11a, matching the training and query distributions consistently yields the lowest latency, reflecting the benefit of workload-specific optimization. When the distributions differ, latency increases by no more than 4 ms. This limited degradation is because LETI considers not only the query workload but also the stable trajectory distribution when learning the traversal order. Overall, LETI benefits from in-distribution training while remaining robust to workload shifts.

6.8 Incremental Insertion

We evaluate incremental insertion on both datasets. LETI is first constructed using 80% of the trajectories. The remaining 20% are then inserted without rebuilding the compact quadtree or retraining the traversal. We compare this incrementally maintained index with the initial index and an index rebuilt using the complete dataset. The same query workload is used for all three settings on each dataset.

As shown in Fig. 11b, incremental insertion increases latency from 21.4 to 29.3 ms on T-Drive and from 50.7 to 69.3 ms on CD-Taxi. This increase is partly due to the additional trajectories, which produce more candidates for scanning and refinement. Moreover, inserting new trajectories changes their distribution in the existing key space, while the original hierarchy and traversal remain unchanged.

Rebuilding the index reduces latency to 23.4 and 61.2 ms, respectively, which is approximately 20% and 12% lower than incremental maintenance. Thus, LETI supports incremental updates without an immediate rebuild, while periodic rebuilding can recover query performance after substantial insertions.



(a) Distribution shift. (b) Incremental insertion.

Fig. 11 Query latency under distribution shift and incremental insertion.

7 Related Work

7.1 Trajectory Data Management Systems

Trajectory data management has been studied in both distributed and database systems. Systems such as HadoopTrajectory [3] and TrajSpark [45] leverage Hadoop and Spark for scalable trajectory analytics, while Distributed MobilityDB [4] extends MobilityDB with distributed query capabilities for spatiotemporal data.

To accelerate query processing, prior work explores specialized indexing and query optimization techniques. BT-Tree [12] organizes trajectories using hierarchical spatial partitions, while Dragoon [9] improves efficiency through index-assisted pruning. These approaches typically follow a filter-and-refine paradigm, where spatial approximations (e.g., MBRs) generate candidates before exact verification.

Recent studies also investigate trajectory indexing in ordered KV stores. TrajMesa [23] adopts XZ-order-based encodings to map spatial objects into one-dimensional keys for efficient range scans, while UlTraMan [8] supports trajectory queries via space linearization in KV storage.

7.2 SFC-based Spatial and Trajectory Indexing

Space-filling curves (SFCs) are widely used to linearize multi-dimensional data into one-dimensional keys for indexing in ordered KV stores [20], supporting spatial queries via range scans over the key space.

Point-based indexing. Lots of work applies SFCs to point data, mapping coordinates directly to one-dimensional keys. ZM-Index [40] and HMIndex [41] combine Z-order or Hilbert curves with learned models to predict data positions. Instance-Optimal Z-Index [32] refines partitioning and traversal to reduce false positives, while WAZI [33] groups frequently co-accessed points to improve locality. These methods are not directly applicable to trajectories. A common adaptation is to decompose trajectories into points, which introduces redundancy and limits pruning.

Trajectory indexing. Prior work on trajectories adopts object-level encodings, typically based on MBRs combined with XZ-order [5]. GeoMesa [16] implements the XZ2 index, and subsequent methods improve intra-region representation. XZ2+ [23] augments MBR encoding with positional signatures, XZ* [14] refines spatial encoding via region combinations, and TShape [15] incorporates multi-scale shape partitions. These methods reduce redundancy and improve filtering, but their spatial organization is largely determined by predefined partitions, limiting adaptability under skewed data and workloads.

7.3 Learning-based SFC Optimization

To improve adaptability, recent work explores optimizing SFC mappings based on data distribution and query workload.

Analytical approaches. Prior studies [29, 43] analyze the clustering properties of SFCs and their impact on range query performance. The Onion Curve [42] achieves near-optimal clustering for regular query shapes. While these results provide useful insights, they rely on simplified assumptions and are less effective under heterogeneous workloads.

Cost-based selection. These work formulates SFC optimization as selecting a mapping from a predefined candidate set. QUILTS [30] enumerates SFCs using bit merging patterns and selects the one minimizing estimated query cost, while subsequent work [27] improves estimation efficiency. These methods incorporate workload information explicitly, but still rely on a single global mapping across the space.

Learning-based construction. Recent approaches construct SFCs using learning techniques. LMSFC [11] models SFCs as parameterized mappings optimized via SMBO, and BMTree [21] applies reinforcement learning to generate piecewise mappings. Beyond directly optimizing linearization functions, BT-Tree [12] employs reinforcement learning to optimize trajectory indexing and organizes trajectory data using a hierarchical structure tailored for large-scale spatiotemporal workloads. These approaches improve flexibility, but mainly optimize the linearization function itself, without considering trajectory-level encoding or hierarchical structure.

8 Conclusion

This paper proposes LETI, an efficient trajectory indexing framework for KV stores. Extensive experiments on real-world datasets verify the strong performance of LETI. In particular, LETI reduces query latency by 13.9%–42.1% on T-Drive and 4.9%–45.7% on CD-Taxi over the strongest baseline. Moreover, LETI consistently outperforms existing methods on similarity search, top-k queries, and large-scale distributed settings. It also remains robust under query distribution shifts and supports incremental insertion without requiring immediate index rebuilding. In future work, we plan to extend LETI to online workload adaptation and spatio-temporal joint indexing.

Statements and Declarations

Funding This work was supported by the National Natural Science Foundation of China under Grant No. 62572086.

Competing interests The authors have no relevant financial or non-financial interests to disclose.

Code availability The source code is available at https://github.com/Spatio-Temporal-Lab/LETI-RLLSFC.

References

1. Alt, H., Godau, M.: Computing the fréchet distance between two polygonal curves. International Journal of Computational Geometry & Applications **5**(1–2), 75–91 (1995)
2. Apache Software Foundation: Apache HBase. https://hbase.apache.org/ (2025)
3. Bakli, M., Sakr, M., Soliman, T.H.A.: HadoopTrajectory: A hadoop spatiotemporal data processing extension. Journal of Geographical Systems **21**(2), 211–235 (2019). DOI 10.1007/s10109-019-00292-4

4. Bakli, M., Sakr, M., Zimányi, E., Dijk, N., Slot, M.: Distributed MobilityDB: A scalable moving object database management system. ACM Transactions on Spatial Algorithms and Systems **11**(2), 1–39 (2025). DOI 10.1145/3719202
5. Böhm, C., Klump, G., Kriegel, H.P.: XZ-ordering: A space-filling curve for objects with spatial extension. In: Advances in Spatial Databases, *Lecture Notes in Computer Science*, vol. 1651, pp. 75–90. Springer, Berlin, Heidelberg (1999)
6. Browne, C.B., Powley, E., Whitehouse, D., Lucas, S., Cowling, P.I., Rohlfshagen, P., Tavener, S., Perez, D., Samothrakis, S., Colton, S.: A survey of monte carlo tree search methods. IEEE Transactions on Computational Intelligence and AI in Games **4**(1), 1–43 (2012)
7. Ding, S., Li, Z., Zhang, K., Mao, F.: A comparative study of frequent pattern mining with trajectory data. Sensors **22**(19), 7608 (2022)
8. Ding, X., Chen, L., Gao, Y., Jensen, C.S., Bao, H.: UlTraMan: A unified platform for big trajectory data management and analytics. Proceedings of the VLDB Endowment **11**(7), 787–799 (2018). DOI 10.14778/3192965.3192970
9. Fang, Z., Chen, L., Gao, Y., Pan, L., Jensen, C.S.: Dragoon: A hybrid and efficient big trajectory management system for offline and online analytics. VLDB Journal **30**(2), 287–310 (2021)
10. Finkel, R.A., Bentley, J.L.: Quad trees: A data structure for retrieval on composite keys. Acta Informatica **4**(1), 1–9 (1974)
11. Gao, J., Cao, X., Yao, X., Zhang, G., Wang, W.: LMSFC: A novel multidimensional index based on learned monotonic space-filling curves. PVLDB **16**(10), 2605–2617 (2023)
12. Gu, T., Feng, K., Yang, J., Cong, G., Long, C., Zhang, R.: BT-Tree: A reinforcement learning-based index for big trajectory data. Proc. ACM Manag. Data **2**(4) (2024)
13. Hamming, R.W.: Error detecting and error correcting codes. Bell System Technical Journal **29**(2), 147–160 (1950)
14. He, H., Li, R., Ruan, S., He, T., Bao, J., Li, T., Zheng, Y.: TRASS: Efficient trajectory similarity search based on key-value data stores. In: Proc. IEEE Int. Conf. Data Eng. (ICDE), pp. 2306–2318. IEEE, Kuala Lumpur, Malaysia (2022)
15. He, H., Xu, Z., Li, R., Bao, J., Li, T., Zheng, Y.: TMan: A high-performance trajectory data management system based on key-value stores. In: Proc. of ICDE, pp. 4951–4964. IEEE, Utrecht, Netherlands (2024)
16. Hughes, J.N., Annex, A., Eichelberger, C.N., Fox, A., Hulbert, A., Ronquest, M.: GeoMesa: A distributed architecture for spatio-temporal fusion. In: Proc. of SPIE, vol. 9473, pp. 128–140. SPIE, Baltimore, MD, USA (2015)
17. Jaccard, P.: Étude comparative de la distribution florale dans une portion des alpes et du jura. Bulletin de la Société Vaudoise des Sciences Naturelles **37**(142), 547–579 (1901). DOI 10.5169/seals-266450
18. Jiang, J., Yan, K., Xia, X., Yang, B.: A survey of deep learning-based pedestrian trajectory prediction: Challenges and solutions. Sensors **25**(3), 957 (2025)
19. Konda, V.R., Tsitsiklis, J.N.: Actor-critic algorithms. In: Advances in Neural Information Processing Systems (NeurIPS 12), pp. 1008–1014. MIT Press, Cambridge, MA, USA (1999)
20. Lawder, J.K., King, P.J.H.: Using space-filling curves for multi-dimensional indexing. In: Advances in Databases (BNCOD), pp. 20–35. Springer, Berlin, Germany (2000)
21. Li, J., Wang, Z., Cong, G., Long, C., Kiah, H.M., Cui, B.: Towards designing and learning piecewise space-filling curves. PVLDB **16**(9), 2158–2171 (2023)
22. Li, R., He, H., Wang, R., Ruan, S., He, T., Bao, J., Zhang, J., Hong, L., Zheng, Y.: TrajMesa: A distributed NoSQL-based trajectory data management system. IEEE Transactions on Knowledge and Data Engineering **35**(1), 1013–1027 (2023)
23. Li, R., He, H., Wang, R., Ruan, S., Sui, Y., Bao, J., Zheng, Y.: TrajMesa: A distributed NoSQL storage engine for big trajectory data. In: Proceedings of the IEEE International Conference on Data Engineering (ICDE), pp. 2002–2005. IEEE, Dallas, TX, USA (2020). DOI 10.1109/ICDE48307.2020.00224
24. Li, R., He, X., Sun, Y., Jiang, J., Shang, Y., Li, G., Chen, C.: Spatio-temporal keyword query processing based on key-value stores. Data Science and Engineering **10**(1), 98–116 (2025)
25. Li, R., Ruan, S., Bao, J., Li, Y., Wu, Y., Zheng, Y.: Querying massive trajectories by path on the cloud. In: Proc. of SIGSPATIAL, pp. 77:1–77:4. ACM, Redondo Beach, California, USA (2017). DOI 10.1145/3139958.3139996
26. Li, R., Ruan, S., Bao, J., Zheng, Y.: A cloud-based trajectory data management system. In: Proc. of ACM SIGSPATIAL, SIGSPATIAL '17, pp. 94:1–94:4. ACM, Redondo Beach, California, USA (2017)
27. Liu, G., Kulik, L., Jensen, C.S., Li, T., Borovica-Gajic, R., Qi, J.: Efficient cost modeling of space-filling curves. PVLDB **17**(13), 4773–4785 (2024)
28. Mockus, J., Tiesis, V., Zilinskas, A.: The application of bayesian methods for seeking the extremum. Towards Global Optimization **2**, 117–129 (1978)
29. Moon, B., Jagadish, H.V., Faloutsos, C., Saltz, J.H.: Analysis of the clustering properties of the hilbert space-filling curve. TKDE **13**(1), 124–141 (2001)
30. Nishimura, S., Yokota, H.: Quilts: Query-aware space-filling curves. In: Proc. of SIGMOD, pp. 1525–1537. ACM, Chicago, IL, USA (2017)
31. Orenstein, J.A., Merrett, T.H.: A class of data structures for associative searching. In: Proceedings of the 3rd ACM SIGACT-SIGMOD Symposium on Principles of Database Systems, pp. 181–190 (1984)
32. Pai, S.G., Mathioudakis, M., Wang, Y.: Towards an instance-optimal z-index. In: Proceedings of the 4th International Workshop on Applied AI for Database Systems and Applications (AIDB@VLDB 2022), pp. 1–10. VLDB Endowment, Sydney, Australia (2022)
33. Pai, S.G., Mathioudakis, M., Wang, Y.: WaZI: A learned and workload-aware Z-index. In: Proceedings of the 27th International Conference on Extending Database Technology (EDBT), pp. 559–571. Paestum, Italy (2024). DOI 10.48786/edbt.2024.48
34. Sakoe, H., Chiba, S.: Dynamic programming algorithm optimization for spoken word recognition. IEEE Transactions on Acoustics, Speech, and Signal Processing **26**(1), 43–49 (1978)
35. Schulman, J., Moritz, P., Levine, S., Jordan, M.I., Abbeel, P.: High-dimensional continuous control using generalized advantage estimation. In: Proc. of ICLR, pp. 1–14. ICLR, San Juan, Puerto Rico (2016)
36. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms (2017)
37. Sun, Y., Huang, T., Zhang, Z., Li, R., Chen, C., Zheng, Y.: EAR-MM: An efficient adaptive and robust algorithm

for streaming map matching. IEEE Trans. Intell. Transp. Syst. **27**(1), 681–691 (2026)
38. Sun, Y., Meng, F., Li, R., Tang, Y., Chen, C., Zhong, J.: Streaming trajectory segmentation based on stay-point detection. In: DASFAA, pp. 203–213. Springer, Cham (2024)
39. Tian, J., Zhao, Z., Ding, Z.: Next point-of-interest recommendation based on joint mining of spatial–temporal and semantic sequential patterns. ISPRS International Journal of Geo-Information **12**(7), 297 (2023)
40. Wang, H., Fu, X., Xu, J., Lu, H.: Learned index for spatial queries. In: Proceedings of the IEEE International Conference on Mobile Data Management (MDM), pp. 569–574. IEEE, Hong Kong, China (2019)
41. Wang, N., Xu, J.: Spatial queries based on learned index. In: Spatial Data and Intelligence, pp. 245–257. Springer, Cham, Switzerland (2021)
42. Xu, P., Nguyen, C., Tirthapura, S.: Onion curve: A space filling curve with near-optimal clustering. In: Proc. of ICDE, pp. 1236–1239. IEEE, Paris, France (2018)
43. Xu, P., Tirthapura, S.: Optimality of clustering properties of space-filling curves. ACM TODS **39**(2), 1–27 (2014)
44. Yuan, J., Zheng, Y., Xie, X., Sun, G.: T-Drive: Driving directions based on taxi trajectories. In: Proc. of ACM SIGSPATIAL, pp. 99–108. ACM, San Jose, CA, USA (2010)
45. Zhang, Z., Jin, C., Mao, J., Yang, X., Zhou, A.: TrajSpark: A scalable and efficient in-memory management system for big trajectory data. In: Asia-Pacific Web (APWeb) and Web-Age Information Management (WAIM) Joint Conference on Web and Big Data, pp. 11–26. Springer (2017)
46. Zheng, Y.: Trajectory data mining: An overview. ACM Transactions on Intelligent Systems and Technology (TIST) **6**(3), 1–41 (2015)